3 Time Series Forecasting: Cumulative GitHub Stars [20 Marks]

3.1 Dataset Description [3 Marks]

You are provided with M GitHub repositories, each represented by a time series of cumulative star counts:

$$y_t^{(i)} = \text{total stars for repository } i \text{ up to time } t.$$

The task is to forecast future growth trajectories and compare classical vs. deep learning methods. The dataset is present at Dataset Link.

Provided Files

- `stars_data.csv` — timestamps, repository IDs, and star counts.
- `repo_metadata.json` — optional meta-features (language, topic, etc.).

3.2 Single Repository Evaluation:

Choose 2 of the repositories present in the dataset, and perform all of the below tasks for the repositories you have chosen.

3.2.1 Preprocessing and Feature Engineering [4 Marks]

1. Clean and align time series; handle missing points or jumps.
2. Choose whether to work in cumulative or incremental domain:

$$\Delta y_t = y_t - y_{t-1}.$$

3. Look at scaling of your data.
4. Ensure temporal integrity—no leakage from future data.
5. Work in different time domains, visualize your data and look as to how the number of stars progress in the dataset. Consider the differences, creating your train-test split would make, based on how you split your forecasting data.

Suggested Implementation Skeleton

- `prep_stars.py`: data cleaning, feature creation, normalization.
- `split_repos.py`: chronological and repository-level splits.

3.2.2 Forecasting Models [7 Marks]

Fit and compare:

- **Classical:** ARMA.
- **Deep Learning:** small RNN and 1D CNN forecasters
- Note: Explain how youre preparing your data for each of these models. Justify your choice in loss function for this dataset.

Suggested Implementation Skeleton

- `classical.py`: wrapper around statsmodels ARMA/ARIMA.
- `dl_models.py`: simple PyTorch/TensorFlow RNN and CNN implementations.
- `train_models.py`:Training loop/calls for all models and hyperparameter search.

3.2.3 Evaluation Protocol [4 Marks]

Metrics and Evaluation

Evaluate model performance using standard error metrics such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

- For **single-timestep forecasting**, compute these metrics at each prediction horizon to quantify immediate predictive accuracy.
- For **multi-step (increasing-window) forecasting**, autoregressively generate future predictions over extended horizons (e.g., $h = 1, 3, 7, 14, \dots$), and plot the mean prediction error as a function of forecast length to analyze how accuracy varies with longer horizons.

Suggested Deliverables

- `evaluate.py`: backtesting, metric computation, and plotting.
- Summary table of metrics across models and horizons.
- Representative forecast and calibration plots.

3.3 Report and Reproducibility [[2 Marks]

Submit a concise report including:

- Data preparation summary and transformations.
- Model architectures and tuning results.
- Quantitative metrics, forecast visuals, and generalization results.

Ensure full reproducibility with all your scripts.

Marking Rubric (Summary)

- Data preprocessing and handling: 4
- Model implementation and comparison: 7
- Evaluation protocol and metrics: 4
- Clarity and reproducibility: 3
- Report polish and visualization quality: 2

4 Variational Autoencoder (VAE) [35 Marks]

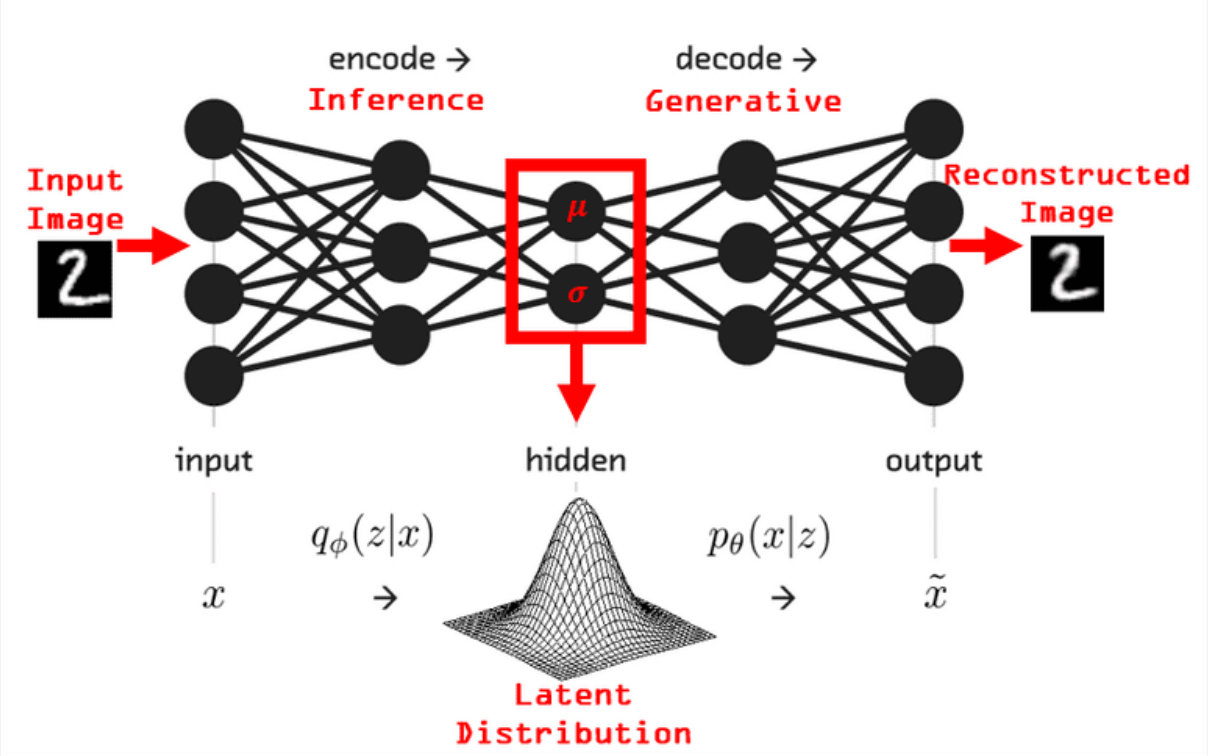


Figure 2: Variational Autoencoder Architecture

4.1 Introduction

A Variational Autoencoder (VAE) is a generative deep learning model that learns to encode data into a continuous latent space and decode it back to reconstruct the input. In this task, you will implement and analyze a VAE trained on the **Fashion-MNIST** dataset. You are allowed to use external libraries for this question.